

UNIVERSIDAD COMPLUTENSE DE MADRID

Máster en Big Data, Data Science &
Inteligencia Artificial

Trabajo fin de máster



Registro de decisiones

Autora: Bolatti, Sofia Huilén

Tutores: Mota, Santiago y
Ortega, Carlos

2026





Registro de decisiones

PARTE I — ETAPA E1: Datos y curación

D1.1 — Arquitectura de la base de datos: SQLite + Parquet

Decisión: los metadatos (muestras, compuestos) se almacenan en SQLite (db/tfm_vocs.db); la matriz de abundancias se almacena por separado en Parquet, un archivo por dataset (db/data/processed/abundancias_<dataset>.parquet).

Motivo: son dos tipos de datos con necesidades distintas, metadatos chicos y categóricos vs. matriz de abundancias que crece y se carga masivamente en pandas para entrenar modelos (formato columnar)

Código: cada loader define DB_PATH = "db/tfm_vocs.db" y PARQUET_PATH = "db/data/processed/abundancias_<dataset>.parquet" como constantes en la sección de configuración, y separa save_to_sqlite() / save_to_parquet() como dos funciones independientes al final del pipeline.

D1.2 — Esquema normalizado en 3 tablas (muestras, compuestos, abundancias)

Decisión: los datos se separan en 3 tablas relacionadas por sample_id/compuesto_id, en vez de una tabla ancha única

Motivo: normalización estándar. Evita repetir metadatos de compuesto por cada muestra que lo mide, y separa formato ancho de formato largo (una fila por observación)

Código: las tres funciones build_muestras(), build_compuestos(), build_abundancias() aparecen, en ese orden, como pasos 2, 3 y 4 de los 5 pasos de cada loader. El "formato largo" de abundancias se arma explícitamente como lista de diccionarios {sample_id, compuesto_id, abundancia, unidad} (long_rows.append({...}))

D1.3 — Un script de carga por dataset, reproducible

Decisión: cada uno de los 6 datasets tiene su propio script load_<dataset>.py en src/, que lee el crudo original y lo deja en las 3 tablas estandarizadas.

Motivo: cada dataset trae un formato de origen distinto por lo que un loader genérico no habría sido viable

Código: los 6 archivos existen y comparten la misma estructura de 5 pasos (leer crudo → build_muestras → build_compuestos → build_abundancias → save_to_sqlite/save_to_parquet → main()), pero cada uno resuelve la interpretación del archivo de origen con lógica propia, por ejemplo load_lazazzara2018.py usa una expresión regular (SAMPLE_COL_PATTERN = re.compile(r"^(\\d+)_(.+)(\\d+)dpi_(\\d+)\\.cmp\$"))



para descomponer nombres de columna tipo "1_Pinot Noir 0dpi_1.cmp", mientras que load_gosh2022.py itera un diccionario SPECIES_CONFIG con las columnas de muestra ya listadas a mano por hoja de Excel, porque ese archivo no tiene un patrón legible por regex.

D1.4 — Estandarización de compuestos por PubChem CID

Decisión: la identidad química de cada compuesto se resuelve contra PubChem, no contra el nombre tal como aparece en cada estudio

Motivo: un mismo compuesto aparece con nombres distintos entre estudios

Código: 3 estrategias según lo que cada archivo crudo ofrecía:

- Por **CAS**: load_lazazzara2018.py (PUBCHEM_CID_BY_INDEX, indexado por posición de fila ya que el Excel no tiene una columna de índice estable) y load_gosh2022.py (CAS_TO_PUBCHEM_CID, diccionario CAS→CID).
- Por **nombre**: load_laupheimer2024.py (PUBCHEM_CID_BY_NAME) y load_Moreira2024.py (PUBCHEM_CID_BY_NAME), porque no traen CAS
- Por **índice interno del propio archivo**: load_Alinc2026.py (PUBCHEM_CID_BY_N). Ese archivo tiene dos picos cromatográficos distintos con el mismo nombre de compuesto, y mapear por nombre habría fusionado dos entradas químicamente distintas en una sola
- load_Ayelo2026.py usa un diccionario VOC_INFO que ya combina nombre + CID en un solo lugar, porque el crudo solo trae columnas genéricas VOC1...VOC13 sin nombre químico propio (el nombre real se reconstruyó a mano contra el paper)

D1.5 — Identificación de los 6 archivos reales

Decisión: data/raw/ contiene exactamente 6 subcarpetas (Alinc, Ayelo, Ghosh, Laupheimer, Lazazzara, Moreira), cada una con un único archivo rawdata_*.

Motivo: las carpetas originales incluían además bioensayos de comportamiento, figuras y resultados estadísticos

Código: cada loader apunta a una ruta fija de un único archivo.

D1.6 — Correcciones de supuestos previos sobre 2 datasets

Decisión: Ayelo2026 no es un ensayo de PGPR, sino un ensayo de resistencia de cultivar de algodón frente a herbivoría por *Spodoptera exigua* y Moreira2024 mide VOCs florales, no foliares

Motivo: Error en el loader

Código: load_Ayelo2026.py yano menciona PGPR en ningún lado; TISSUE es "leaves" (foliar) y el diseño biológico es cultivar (resistente/susceptible) × herbivoría por 5 poblaciones de *S. exigua*, colapsadas en physiological_state = "herbivory_exposed". load_Moreira2024.py define TISSUE = "flowers (floral VOCs)"



D1.7 — Bugs corregidos durante la carga

Decisión: se encontraron y corrigieron dos errores en `load__laupheimer2024.py` y en la tabla compuestos compartida:

1. Error de indentación
2. Columnas faltantes en compuestos (`dataset_origin`, luego `compound_class/species`), corregidas con `ALTER TABLE` sin pérdida de datos.

Código: `load__laupheimer2024.py` ya define correctamente `def save_to_sqlite(muestras, compuestos, db_path):` con indentación consistente de 4 espacios en todo `main()`

D1.8 — Duplicación de datos (detección y corrección)

Decisión : `Lazazzara2018` se había cargado dos veces; `Laupheimer2024` se duplicó parcialmente. Se eliminaron los duplicados por clave única. Resultado final: 294 muestras / 297 compuestos.

Motivo: los loaders usan `to_sql(..., if_exists="append")`, lo cual no protege contra reinserción si el script se corre más de una vez sin verificar antes

D1.9 — Resolución de PubChem CID para Ghosh2022

Decisión: 133 se resolvieron directo, 4 tenían errores de tipeo en el CAS original, corregidos manualmente. El mapeo se integró directo en el loader

Código: el diccionario `CAS_TO_PUBCHEM_CID` está hardcodeado dentro de `load_gosh2022.py`, no en un script aparte. El mapeo se aplica con `.get(row["Cas No"])` (devuelve `None` si no hay match, en vez de fallar)

D1.10 — Unificación de esquema: `eje_biologico` y `metadata_extra`

Decisión: se agregan dos columnas nuevas a muestras (`eje_biologico`, `metadata_extra` en JSON) y se fusionan ahí las 4 tablas que habían quedado separadas (`muestras_alinc2026`, `muestras_ayelo2026`, `muestras_moreira2024`, `muestras_ghosh2022`).

Motivo: cuatro de los seis loaders introducen columnas biológicas que el esquema común de muestras no tenía, se dejaron en tablas propias hasta esta etapa de unificación.

Código, `src/01_unify_schema.py`:

- `COLUMNAS_MUESTRAS_ACTUALES`: set de 17 columnas que sí existen en muestras
- `TABLAS_OUTLIER`: diccionario que mapea cada tabla separada a su `eje_biologico`.
`muestras_alinc2026` → `"herbivoria_biocontrol"`, `muestras_ayelo2026` → `"resistencia_cultivar_herbivoria"`, `muestras_moreira2024` → `"herbivoria"`, `muestras_ghosh2022` → `"infeccion_viral"`.
- `asegurar_columnas_nuevas()`: agrega `eje_biologico` y `metadata_extra` a muestras vía `ALTER TABLE` solo si no existen ya



- `backfill_estandar()`: en las filas ya existentes (Lazazzara/Laupheimer) sin `eje_biologico`, lo completa con 'estandar'.
- `fusionar_tabla_outlier()`: por cada fila de una tabla outlier, separa sus columnas en dos grupos: las que coinciden con `COLUMNAS_MUESTRAS_ACTUALES` van directo a columnas de muestras; el resto se empaqueta como JSON en `metadata_extra` (`json.dumps(extra, ensure_ascii=False, default=str)`). No borra las tablas originales
- El script es ejecutable directo por línea de comandos: `src/01_unify_schema.py db/tfm_vocs.db`, e imprime un resumen final (`SELECT COUNT(*) FROM muestras, agrupado por eje_biologico y por species`)

`eje_biologico` se diseñó inicialmente como posible variable objetivo del clasificador, pero en la Etapa E2 se determinó que su función real es solo de metadato de curación

D1.11 — Cada loader decide por sí mismo si escribe en `muestras` o en `muestras_<dataset>`

Decisión: `load_lazazzara2018.py` y `load__laupheimer2024.py` escriben directo en la tabla común `muestras` (`to_sql("muestras", ..., if_exists="append")`), porque su diseño experimental (2 estados, sin ejes biológicos adicionales) cabe en el esquema común desde el inicio. Los otros 4 loaders escriben en una tabla propia (`to_sql("muestras_<dataset>", ..., if_exists="replace")`)

Motivo: evita forzar un esquema común antes de tener claro qué columnas hacen falta, en vez de ir parchando el esquema común loader por loader

D1.12 — Tratamiento de valores no detectados

Decisión: cada loader preserva el criterio de no detectado tal como lo maneja su archivo de origen

- `load_Alinc2026.py`: guarda `0.0` explícito (`abundancia = float(valor) if valor is not None else 0.0`) porque el Excel de origen ya usa "0" para picos no detectados
- `load__laupheimer2024.py`: trata celda vacía o " " como `None` (`if valor == " " or valor == "" or pd.isna(valor): abundancia = None`).
- El resto de los loaders no tiene valores faltantes en su crudo (`load_Moreira2024.py` y `load_Ayelo2026.py`).

D1.13 — La unidad de abundancia no se normaliza en E1

Decisión: cada loader escribe su propia unidad de abundancia tal como la reporta el estudio, en la columna `unidad` de la tabla `abundancias`, sin convertir entre unidades

Motivo: los 6 estudios miden con técnicas distintas y reportan en unidades no comparables directamente entre sí

D1.14 — Ghosh2022: separación tomate/pimiento

Decisión: `load_gosh2022.py` procesa las dos especies como configuraciones independientes desde la extracción misma. `SPECIES_CONFIG` tiene una entrada por especie, cada una apuntando a



su propia hoja de Excel ("3W tomato VOCs" / "3W pepper VOCs") y su propio set de columnas de muestra ,aunque en esta etapa ambas todavía se concatenan (`pd.concat`) en una sola tabla `muestras_ghosh2022` y un solo `abundancias_ghosh2022.parquet`, distinguibles por la columna `species`.

D1.15 — Colapso de poblaciones de herbivoría en Ayelo2026

Decisión: el colapso de las 5 poblaciones geográficas de *Spodoptera exigua* en un único estado `physiological_state = "herbivory_exposed"` ocurre directamente en `build_muestras()`, conservando la población específica en la columna `treatment_group` (no se pierde el dato, solo se colapsa la variable objetivo)

D1.16 — Convención de identificadores

Decisión: tanto `sample_id` como `compuesto_id` se generan por concatenación de campos legibles del propio dato (`dataset + tratamiento/genotipo + timepoint + réplica`, con ceros a la izquierda vía `.zfill()`),

Motivo: permite inspeccionar la base de datos a simple vista y detectar errores de carga sin tener que hacer join contra una tabla de metadato



PARTE II — ETAPA E2: Clasificación baseline

D2.1 — Variable objetivo del clasificador

Decisión: la variable objetivo es `physiological_state`. La especie queda resuelta por diseño (un modelo por especie), salvo en Ghosh2022 donde se combina tomate+pimiento y la especie pasa a ser covariable

Motivo: la propuesta define el objetivo específico como clasificación del estado fisiológico

Código: en las 6 secciones, `y = datos_modelo["physiological_state"]`. En Ghosh2022, `species` se incorpora como columna de `X` vía `pd.get_dummies(..., columns=["species"], drop_first=True)` pero como covariable

D2.2 — Corrección de la etiqueta de estado fisiológico de Ayelo2026

Decisión: se corrigió `physiological_state` de "tratado con PGPR" a "expuesto a herbivoría".

Código: confirmado en `load_Ayelo2026.py`, el notebook de E2 consume `physiological_state` ya corregido desde la base; no hay lógica de corrección dentro del propio notebook de E2

D2.3 — Manejo de clases con muy pocas muestras

Decisión: se excluyó del análisis cuantitativo la combinación con 4-5 muestras por clase (Alinc2026, biocontrol + herbivoría en tomate)

Código: la sección de Alinc2026 no tiene `train_test_split`, `GridSearchCV` ni comparación de algoritmos, pasa directo de la matriz de features a un único `RandomForestClassifier` entrenado sobre el 100% de los datos

D2.5 — Primero intento de clasificador único, finalmente análisis por especie

Decisión: se eliminó el diseño de un único modelo combinando las 6 especies (9 clases combinando especie + estado). Se reemplazó por modelos independientes

Motivo: al poner en práctica el modelo único, se detectó que el algoritmo aprendía a distinguir de qué estudio provenía cada muestra en lugar de aprender el estado real de la planta

Código: `02_EDA_baselinemuliespecie_ELIMINADO.ipynb`

1. Análisis de matriz de confusión del árbol base: "infected (cebada+vid, biotrofos) se confunde masivamente con control (11/13). Agrupar dos especies distintas bajo un mismo estado puede ser demasiado heterogéneo.
2. Random Forest: el modelo estaba usando una columna indicadora binaria de "compuesto medido o no" en vez de la abundancia real, es decir, aprendía qué estudio mide qué compuestos. Se reentrena excluyendo esas columnas (`cols_sin_indicador = [c for c in X_train_rf.columns if not c.endswith("_medido")]`)).
3. Se probó restringir el modelo a los 31 compuestos medidos en 2 o más datasets, como intento de eliminar el efecto de lote.



4. Al revisar compuesto por compuesto si de verdad tenían cobertura en el par específico de estudios que sostenía una comparación puntual (control/infected = Lazazzara2018 vs. Laupheimer2024), aparecen compuestos como CID 244, 957, 998 medidos solo en Lazazzara2018 (98 muestras) y CID 2758, 4133 medidos solo en Laupheimer2024 (32 muestras), cero solapamiento real en ese par, pese a figurar en la lista de compartidos

D2.6 — Normalización de abundancias

Decisión: estandarización (z-score) de cada compuesto, calculada por separado dentro de cada especie

Código: cada sección del notebook filtra muestras/abundancias por dataset_origin antes de calcular `matriz_features.apply(zscore)`. El z-score nunca se calcula sobre una matriz que mezcle datasets. Excepción con matiz técnico en Ghosh2022, ahí el z-score se calcula dos veces con lógica distinta según el algoritmo destino

D2.7 — Selección del modelo final

Decisión: en cada uno de los 6 análisis, la elección final se basó en la consistencia del modelo entre particiones (boxplot de los 5 folds de CV de los 5 mejores candidatos), no solo en el mejor promedio automático de GridSearchCV.

Código (con una celda de justificación en cada caso, para las 5 especies con comparación):

- **Vid:** *res_1 (entropy, max_depth=5, min_samples_leaf=3, min_samples_split=10, n_estimators=200)*. Tiene el peor caso más alto que el resto (0.60 vs 0.52), y aunque su techo sea un poco menor, prefiero un modelo más parejo entre folds. **Modelo final:** Me quedo con el **árbol simple**. Ni RF (0.798) ni XGBoost (0.792) le ganan al árbol (0.80), un modelo más complejo no suma
- **Cebada:** Los 5 candidatos son equivalentes, elijo *res_1*. **Modelo final:** Se selecciona el **árbol de decisión simple**, el cual empatía en rendimiento con Random Forest, por parsimonia
- **Colza:** Elijo *res_5m*. Misma mediana que *res_1* a *res_4*, pero con mejor techo y menos dispersión entre folds. **Modelo final:** **árbol de decisión**. No por superioridad de rendimiento ya que todos resultaron prácticamente equivalentes. El techo de rendimiento en este dataset está determinado por el tamaño de muestra disponible
- **Algodón:** Debido a que son todos iguales, elegí *res_1a* (RF) y *res_1xa* (XGB). **Modelo final:** Se selecciona **Random Forest** como modelo final, superando claramente tanto al árbol de decisión simple como a XGBoost tuneado
- **Ghosh (tomate+pimiento):** Debido a que son todos iguales, selecciono *res_1g* (RF) y *res_1xg* (XGB). **Modelo final:** Se selecciona **XGBoost** como modelo final. Esta diferencia se explica por el manejo nativo de valores faltantes de XGBoost

En los 5 casos, `random_state=123` fijo en el estimador de GridSearchCV hace que "res_1" a "res_5" sean reproducibles entre corridas

D2.8 — Detección de un contaminante de laboratorio

Decisión: se excluyó un ftalato que apareció como el tercer predictor más importante en el SHAP de vid

Motivo: los ftalatos no son metabolitos vegetales



Código : el hallazgo salió del SHAP (CID 6782 con importancia 0.114, tercer lugar). Se excluye a mano (`cols_sin_ftalato = [c for c in X_train.columns if c != "6782"]`) y se reentrena. Se agrega, después del hallazgo, un chequeo sistemático por regex ("`phthalate|siloxane|silane|BHT|butylated hydroxytoluene`") para confirmar que no quedan más contaminantes. Ese mismo chequeo se copia luego al inicio de las otras 5 secciones, antes de construir la matriz de features

D2.9 — Descarte de LeaveOneOut para el dataset más chico

Decisión: para Alinc202, se optó por un análisis descriptivo sin partición de datos, en vez de LeaveOneOut

Código:

```
modelo_descriptivo_alinc = RandomForestClassifier(

    n_estimators=100, max_depth=5, min_samples_leaf=2,

    class_weight="balanced", random_state=123

)
```

```
modelo_descriptivo_alinc.fit(X_completo, y_completo)
```

El SHAP también se calcula sobre `X_completo`, ya que el objetivo es describir qué compuestos separan las clases dentro de los datos observados. `export_classifier()` guarda explícitamente "`f1_macro_test`": None en los metadatos, para que se sepa que este modelo no tiene una métrica de test comparable a los otros 5

D2.11 — Ghosh2022: dos matrices de features distintas

Decisión: se construyen dos versiones de la matriz estandarizada `matriz_zscore_rf` (NaN imputados a 0 antes del z-score, vía `scipy.stats.zscore`) y `matriz_zscore_xgb` (NaN preservados, calculada a mano como `(matriz_features - matriz_features.mean()) / matriz_features.std()`, porque `scipy.stats.zscore` no maneja NaN y devuelve la columna entera en NaN si encuentra uno)

Motivo: Ghosh022 tiene 39.6% de NaN estructural. Random forest no acepta NaN como input, así que necesita imputación; XGBoost sí maneja NaN y puede aprender una dirección óptima para los valores ausentes (`tree_method="hist"`), por lo que imputarlos de entrada le quita información.

D2.12 — species como covariable en Ghosh2022

Decisión: `species` se incorpora a `X` vía `pd.get_dummies(datos_modelo, columns=["species"], drop_first=True)`, generando una columna `species_Solanum lycopersicum`



D2.13 — random_state=123 fijo en todo el notebook

Decisión: las 6 secciones fijan random_state=123 en train_test_split, DecisionTreeClassifier, RandomForestClassifier, el estimador base de cada GridSearchCV, y XGBClassifier.

Motivo: sin esto, las elecciones dejarían de tener sentido en una re-ejecución, ya que dependen de comparar resultados numéricos concretos entre 5 candidatos, y esos números solo son estables si toda la aleatoriedad del pipeline está fijada con la misma semilla.

D2.14 — Los clasificadores se exportan a disco

Decisión: el notebook 02_EDA_baseline_por_especie.ipynb define una función export_classifier() que guarda a disco (models/<especie>/, vía joblib) el modelo, las columnas de features, el label encoder y un diccionario de metadatos, y la llama al final de las 6 secciones, con MODELS_DIR = Path("../models").



PARTE III — ETAPA E3: CVAE

D3.1 — Organización del código: notebook, sin módulos en src/

Decisión: el código de E3 se escribe directamente en las celdas sin extraer funciones a src/ como en E1/E2.

Código: los 6 notebooks no importan ningún módulo propio del repositorio, solo tensorflow, sklearn, pandas, numpy, pathlib, pickle. La única función reutilizable (export_cvae) está definida dentro del propio notebook

D3.2 — Un CVAE independiente por especie

Decisión: se entrena un CVAE por especie (Ghosh2022 se parte en dos)

Código: cada notebook carga sus datos con `WHERE dataset_origin = '<Dataset>'` y construye su propio wide, `X_log`, encoder/decoder/CVAE desde cero

D3.3 — Preprocesamiento con log1p + StandardScaler + get_dummies

Decisión: las abundancias se transforman con log1p y se estandarizan con StandardScaler (fit en train); la condición (physiological_state) se codifica con `pd.get_dummies(drop_first=True)`.

Código: idéntico en las 7 secciones. `X_log = np.log1p(wide.values)` seguido de `scaler.fit_transform(X_train) / scaler.transform(X_val)`; `cond_dummies = pd.get_dummies(estado_cat, drop_first=True)`

D3.4 — Arquitectura condicionada con warm-up de KL

Decisión: encoder-decoder denso con la condición concatenada en ambas mitades; pérdida = reconstrucción + KL con warm-up de beta (0→1 en 50 épocas)

Código: idéntico patrón en las 7 secciones

```
class KLWarmup(keras.callbacks.Callback):

    def __init__(self, cvae, warmup_epochs=50, max_beta=1.0):

    def on_epoch_begin(self, epoch, logs=None):

        new_beta = self.max_beta * min(1.0, epoch / self.warmup_epochs)

        self.cvae.beta.assign(new_beta)
```

Es una suma de cuadrados por muestra, promediada luego sobre el batch



```
recon_loss = tf.reduce_mean(tf.reduce_sum(tf.square(x - x_recon), axis=1))
```

(reduce_sum sobre axis=1, es decir sobre los compuestos, y recién después reduce_mean sobre las muestras). Es una diferencia de escala frente a un MSE por elemento, con input_dim compuestos, esta pérdida es aprox input_dim veces más grande que el MSE equivalente.. No afecta el entrenamiento, pero sí explica por qué los valores de recon_loss que aparecen en los notebooks parecen altos para lo que uno esperaría de un MSE estandarizado

D3.5 — Corrección: propiedad metrics faltante en la clase CVAE

Decisión: se agregó la propiedad metrics (con loss_tracker, recon_loss_tracker, kl_loss_tracker) a la clase CVAE.

Código: En las 7 definiciones de CVAE de los 6 notebooks

@property

```
def metrics(self):  
  
    return [self.loss_tracker, self.recon_loss_tracker, self.kl_loss_tracker]
```

D3.6 — Reducción de capacidad del modelo por overfitting (Vid)

Decisión: arquitectura reducida de 32→16 a 16→8 en las capas ocultas de encoder y decoder.

Código: el estado final del notebook (post-decisión) muestra layers.Dense(16, activation="relu") → layers.Dense(8, activation="relu") → z_mean/z_log_var, simétrico en el decoder.

D3.7 — Extensión del entrenamiento a 800 épocas (Vid)

Decisión: epochs=800, batch_size=16.

Código: epochs=800, batch_size=16. En Laupheimer2024 (misma arquitectura base, batch_size=8 en vez de 16)

D3.8 — Evaluación del espacio latente sobre datos de validación

Decisión: el silhouette score se calcula sobre z_mean_val (validación), no sobre entrenamiento

Código: sil = silhouette_score(z_mean_val, labels_val), mismo patrón (z_mean_val/z_val) en las 7 secciones.

D3.9 — Separabilidad del espacio latente por condición

Decisión: se documenta que el silhouette score de z por physiological_state es bajo y no se interpreta como una falla del modelo



Código: Silhouette score (validación): 0.109.

D3.10 — Exploración del gradiente: interpolación de la condición con z fijo (Vid)

Decisión: en vez de interpolar entre dos puntos de z, se fija z en el origen y se interpola la condición de 0 (control) a 1 (infected) en 11 pasos, decodificando en cada paso

Código: `z_fixed = np.zeros((1, latent_dim))`, seguido de un barrido de la condición reconstruido en el decoder. Compuesto más relevante documentado: delta-Cadinene, coincidente con el discriminador principal de SHAP en E2

D3.11 — Imputación de valores no detectados (Laupeimer2024)

Decisión: los NaN se imputan como 0 antes de aplicar $\log_{1p}$.

Código: patrón `wide = wide.fillna(0)` con impresión previa/posterior de nulos, el mismo patrón está en Ghosh2022

D3.13 — Arquitectura 16→8 aplicada directamente (Laupheimer2024)

Decisión: arquitectura simplificada (16→8, `latent_dim=2`) desde el inicio, sin repetir el ciclo de diagnóstico de overfitting

D3.16 — Condición categórica de 3 niveles (Alinc2026)

Decisión: `pd.get_dummies(drop_first=True)` sobre una variable categórica con orden explícito (`control`, `herbivory_only`, `biocontrol_herbivory`), forzando a `control` como referencia. Resultan 2 columnas dummy

Código:

```
estado_cat = pd.Categorical(estado, categories=["control", "herbivory_only",
"biocontrol_herbivory"])
```

```
cond_dummies = pd.get_dummies(estado_cat, drop_first=True).astype(float)
```

Columnas de condición: `['herbivory_only', 'biocontrol_herbivory']`

D3.17 — Split 0.33 por tamaño muestral chico (Alinc2026)

Decisión: `test_size=0.33` estratificado (10 train / 5 val).

Código: `train_test_split(X_log, cond_dummies.values, test_size=0.33, stratify=estado, random_state=42) → X_train: (10, 23) X_val: (5, 23)`.

D3.18 — Arquitectura reducida a 8→4 (Alinc2026)

Decisión: encoder/decoder de 8→4 unidades ocultas.



Código: `layers.Dense(8, activation="relu") → layers.Dense(4, activation="relu") → z_mean/z_log_var`

D3.19 — Exploración por gradiente adaptada a 3 condiciones (Alinc2026)

Decisión: dos barridos independientes desde `control` (`control→herbivory_only` y `control→biocontrol_herbivory`), cada uno fijando la otra dummy en 0

Código: Dos bloques etiquetados `"condicion"`: `"herbivory_only"` y `"condicion"`: `"biocontrol_herbivory"` construidos por separado

D3.21 — Separación de Ghosh2022 en dos modelos por especie

Decisión: el dataset se divide en dos CVAE independientes (tomate y pimiento) dentro del mismo notebook

Código: tiene dos bloques de preprocesamiento/entrenamiento completos y paralelos, el primero usa variables sin sufijo (`estado`, `X_train`, `latent_dim`), el segundo usa sufijo `_p` de "pimiento" (`estado_p`, `X_train_p`, `z_mean_p`).

D3.22 — Relación dimensiones/muestras desfavorable, arquitectura mínima (Ghosh2022)

Decisión: arquitectura de una sola capa de 4 unidades, aceptada sin más optimización

Código: tomate: `x = layers.Dense(4, activation="relu")(x)` antes de `z_mean`; pimiento: mismo patrón con sufijo `_p`. `X_train`: (10, 76) para tomate (76 compuestos, 10 muestras train) y `test_size=0.33` en ambas partes)

D3.23 — Condición de 3 niveles, misma codificación que Alinc2026 (Ghosh2022)

Decisión: one-hot con `control` como referencia (2 dummies) en ambas partes.

Código: tomate: `estado_cat = pd.Categorical(estado, categories=["control", "herbivory_only", "infected_viral"], columns=resultantes ['herbivory_only', 'infected_viral'])`

D3.24 — Hallazgos biológicos: tomate vs. pimiento (Ghosh2022)

Decisión: se documentan como comparación cualitativa: tomate muestra inducción de 2-Hexenal (verde/daño mecánico) con `herbivory_only` y supresión general con `infected_viral`; pimiento muestra un mismo grupo de compuestos respondiendo en direcciones opuestas según el tipo de estrés.

Código :

- Tomate: Silhouette score (validación): 0.0037
- Pimiento: Silhouette score (validación): 0.017



D3.25 — Condición de 5 niveles (Moreira2024)

Decisión: one-hot con control como referencia (4 dummies): control + 2 tipos de herbivoría (oruga, pulgón) + 2 tipos de infección (bacteriana, fúngica).

Código:

```
estado_cat = pd.Categorical(estado, categories=["control", "herbivory_caterpillar",  
"herbivory_aphid", "infected_bacterial", "infected_fungal"])
```

```
cond_dummies = pd.get_dummies(estado_cat, drop_first=True).astype(float)
```

```
Columnas de condición: ['herbivory_caterpillar', 'herbivory_aphid',  
'infected_bacterial', 'infected_fungal']
```

D3.26 — Mejor relación dimensiones/muestras de todas las especies (Moreira2024)

Decisión: arquitectura estándar 16→8 directamente, sin ciclo de simplificación

Código: (16→8→latent); X_train: (48, 15) (15 compuestos, 60 muestras totales)

D3.27 — 300 épocas en vez de 800 (Moreira2024)

Decisión: se reduce el entrenamiento a 300 épocas

Código: EPOCHS = 300

D3.28 — Hallazgo biológico: supresión floral generalizada + benzaldehído como diferenciador (Moreira2024)

Decisión: se documenta como hallazgo transversal entre los 4 tipos de estrés, la mayoría de compuestos del top-5 bajan de forma consistente en los 4 barridos (trade-off defensa/atracción de polinizadores); el benzaldehído es la excepción (sube con infección bacteriana, baja con infección fúngica).

Código: Silhouette score (validación): -0.296

D3.29 — Cultivar de resistencia como variable de fondo (Ayelo2026)

Decisión: la condición del CVAE es únicamente physiological_state (control/herbivory_exposed); el cultivar (RC resistente / SC susceptible) no se incorpora al vector de condición

Código: cond_dummies = pd.get_dummies(estado, drop_first=True).astype(float)
→ Columnas de condición: ['herbivory_exposed']

D3.30 — Split estratificado con clases desbalanceadas (Ayelo2026)

Decisión: test_size=0.2 estratificado, manteniendo la proporción 5:1 (control:herbivory_exposed) en train y validación.



Código: `train_test_split(X_log, cond_dummies.values, test_size=0.2, stratify=estado, random_state=42)` → Distribución train: {1.0: 40, 0.0: 8} (5:1), Distribución val: {1.0: 10, 0.0: 2} (5:1)

D3.31 — Mejor resultado de ajuste de las 6 especies (Ayelo2026)

Decisión: se documenta como referencia de "mejor caso". 13 compuestos, 60 muestras (48 train), menor ratio de overfitting

Código: `X_train: (48, 13) X_val: (12, 13)`, arquitectura estándar 16→8 (igual que Vitis/Laupheimer/Moreira, sin necesidad del ciclo de reducción de Alinc/Ghosh).

D3.32 — Hallazgo biológico: firma HIPV clásica (Ayelo2026)

Decisión: se documenta como validación biológica fuerte. Los compuestos más relevantes del gradiente control→herbivory_exposed son, en su totalidad, VOCs de inducción por herbivoría (HIPVs) ampliamente documentados en la literatura.

Código: `Silhouette score (validación): -0.222`

D3.33 — La pérdida de reconstrucción no es un MSE plano

Es `reduce_mean(reduce_sum(square(x - x_recon), axis=1))`, una suma de cuadrados por muestra promediada sobre el batch, no un promedio por elemento

D3.34 — Ghosh2022 usa batch_size distinto entre sus dos sub-modelos.

Tomate: `batch_size=8`. Pimiento: `batch_size=4`, con el comentario explícito en el código # solo 9 muestras de train.

D3.35 — E3 y E4 comparten notebook y carpeta de salida

Cada `03_CVAE_<dataset>.ipynb` termina exportando, en la misma llamada a `export_cvae()`, tanto los artefactos del CVAE (E3: `cvae_encoder.keras`, `cvae_decoder.keras`) como los del Isolation Forest y autoencoder de reconstrucción (E4: `isolation_forest.pkl`, `autoencoder.keras`, `scores_loo.npy`, `errores_loo_ae.npy`)



PARTE IV — ETAPA E4: Detección de anomalías

D4.1 — Elección de método: Isolation Forest + autoencoder de reconstrucción combinados

Decisión: se usan dos detectores de anomalías no supervisados en paralelo

Código: en las 7 secciones aparece el mismo par `from sklearn.ensemble import IsolationForest` + un autoencoder Keras propio (`build_autoencoder`, definido en cada notebook)

D4.2 — Detectores entrenados independientes del CVAE

Decisión: Isolation Forest y el autoencoder se entrenan desde cero, solo con perfiles reales, sin usar el encoder/decoder del CVAE como criterio de plausibilidad

Código: `iso_final_<especie> = IsolationForest(...)` y `ae_final_<especie> = build_autoencoder(...)` se ajustan (`.fit(X_real_<especie>, ...)`) sobre `X_real_<especie>`, la matriz de datos reales no sobre `X_train/X_val` del CVAE ni sobre reconstrucciones del decoder.

El decoder del CVAE solo se usa para *generar* los perfiles a evaluar, no para entrenar los detectores

D4.3 — Calibración con validación cruzada leave-one-out (LOO)

Decisión: la distribución de referencia se calibra reentrenando el detector N veces, dejando una muestra real afuera cada vez

Código: `from sklearn.model_selection import LeaveOneOut` seguido de `loo = LeaveOneOut()` y un bucle `for train_idx, test_idx in loo.split(...)` que reentrena `iso_fold/ae_fold` en cada iteración.

D4.4 — Clasificación en tres categorías por percentil

Decisión: "plausible" si todas las señales están debajo del percentil 90; "novedoso" si todas superan el percentil 97.5; "frontera" en cualquier otro caso (desacuerdo o zona intermedia).

Código: función `clasificar()` presente en `vid`, `cebada`, `colza` y `algodón`:

```
def clasificar(perc_if, perc_ae=None):
    percentiles = [p for p in [perc_if, perc_ae] if p is not None]
    if all(p > 97.5 for p in percentiles):
        return "novedoso"
    elif all(p < 90 for p in percentiles):
        return "plausible"
```

Esta función `clasificar()` no aparece en Alinç2026 ni Ghosh2022, solo se calculan percentiles de Isolation Forest por separado, sin combinarlos en una etiqueta final.



D4.5 — Control positivo obligatorio por especie

Decisión: antes de clasificar los perfiles generados, se valida que ambos detectores distingan un perfil anómalo, por especie

Código: las 7 secciones tienen una celda `# Control + para corroborar / # Control positivo` que construye `z_extremo_<especie>` y evalúa `score_if_control_<especie> / error_ae_control_<especie>` antes de cualquier clasificación

D4.6 — Dos tipos de control positivo: z extremo (vía decoder) y perfil barajado (independiente del decoder)

Decisión: control z alejado del origen decodificado; control valores de compuestos de una muestra real barajados al azar

Código: `control z_extremo_<especie> = np.full((1, latent_dim), 5.0)` seguido de `decoder.predict(...)`, presente en las 7 secciones. `Control rng = np.random.default_rng(0), perfil_barajado = X_real_alinc[0].copy(), rng.shuffle(perfil_barajado)`

D4.8 — Alinc2026: detectores sin poder estadístico, tratado como cualitativo

Decisión: ni Isolation Forest ni el autoencoder tienen poder de detección confiable para este dataset; se trata como caso cualitativo

Código — control estándar: Score Isolation Forest: (percentil 60.0%), Error autoencoder: (percentil 20.0%). Ninguno cerca del percentil 90 que definiría una clasificación con confianza.

Código — prueba de magnitud creciente:

z=5: IF 60.0% AE 20.0%
z=10: IF 60.0% AE 80.0%
z=20: IF 66.7% AE 100.0%
z=50: IF 73.3% AE 100.0%

Isolation Forest nunca supera 73.3% ni con z=50 confirmado como saturación del decoder

Código — control independiente del decoder: IF percentil: 60.0%, AE percentil: 60.0% tampoco supera el percentil 90.

Código — ajustes de detector probados: `IsolationForest(n_estimators=500, max_features=0.5, ...)` y `PCA(n_components=3, ...)` como autoencoder lineal alternativo, ambos recalibrados con LOO

D4.9 — Ghosh2022 (tomate y pimiento) confirmado de forma independiente

Decisión: ambas partes de Ghosh2022 se tratan como caso cualitativo



Código: nb_03_ghosh2022 reproduce, en ambas partes, la misma secuencia que Alinc2026: control estándar, control independiente del decoder, y los mismos ajustes de detector (`IsolationForest(n_estimators=500, max_features=0.5, ...)`; `PCA(n_components=3, ...)`), sin ningún llamado a `clasificar()` en ninguna de las dos partes.

D4.10 — Resultado Lazazzara2018: plausible en todo el rango evaluado

Decisión : los 22 perfiles se clasifican como "plausible" en su totalidad.

Código: Score Isolation Forest: 0.5828 (percentil 100.0%), Error autoencoder: 3.2404 (percentil 100.0%)

Hallazgos metodológicos transversales

- La potencia de un detector de anomalías no es una propiedad fija del método, depende de la relación compuestos/muestras de cada dataset
- Un control positivo con z extremo puede fallar por saturación del decoder (Alinc2026); el control independiente del decoder es la prueba más confiable para decidir si confiar en un resultado de plausibilidad



PARTE V — ETAPA E5: Dashboard interactivo

D5.1 — Elección Flask+Plotly

Decisión: entre las alternativas, se elige Flask+Plotly.

Motivo: las rutas Flask mapean 1 a 1 con los 6 módulos; da control total de estado, necesario para el Módulo 6 (reentrenamiento por confirmación humana).

D5.2 — Eportar modelos a disco antes de tocar el dashboard

Decisión: se exportan a `models/` todos los artefactos de las Etapas 2, 3 y 4 antes de escribir código de interfaz

Motivo: Un dashboard no puede reentrenar 7 CVAEs cada vez que alguien lo abre.

D5.4 — `models/` se versiona en git, a diferencia de `data/raw`

Decisión: `models/` se comitea normalmente, sin `.gitignore`.

Motivo: a diferencia de `data/raw` (excluido por tamaño) y los `.parquet` procesados (regenerables), los artefactos de modelo son chicos e indispensables para que cualquiera que clone el repo pueda abrir la app sin reentrenar

D5.5 — Consistencia del `.gitignore`: CSVs procesados también se excluyen

Decisión: se agrega `db/data/processed/*.csv` al `.gitignore`.

Motivo: son el mismo contenido que los `.parquet` ya ignorados. misma lógica de "regenerable desde los loaders".

D5.7 — Ausencia de detector en 3 especies

Decisión: `alinc2026`, `ghosh2022_tomate` y `ghosh2022_pimiento` no tienen `isolation_forest.pkl`/`autoencoder.keras/arrays LOO` de forma intencional.

Motivo: Ambos detectores se probaron sin alcanzar confianza suficiente por la relación desfavorable compuestos/muestras. Se reflejan como "caso cualitativo" vía la clave `estado_etapa4` en `cvae_meta.pkl`.

D5.8 — Entorno de ejecución: Python 3.12 en entorno virtual separado

Decisión: el dashboard corre en `C:\venvs\tfm-vocs` (Python 3.12).

Motivo: TensorFlow no tiene distribución para 3.14. Se adoptó invocar siempre el intérprete por ruta completa en vez de depender de la activación

D5.9 — Estructura de la app: separación de datos y rutas

Código : `app.py` importa todo desde `data_access.py` (`CLASSIFIER_SPECIES`, `CVAE_SPECIES`, `CATEGORIAS_CONDICION`, `load_eda_data`, ...) y no contiene ninguna consulta SQL ni carga de modelo directamente. `CLASSIFIER_SPECIES` (6 entradas) y `CVAE_SPECIES` (7 entradas, con `ghosh2022_tomate/ghosh2022_pimiento` separados) confirmadas en `data_access.py`.



D5.10 — Módulo 1 (EDA)

Código: `app.py`, ruta `/eda` — top5 calculado con `.groupby("compuesto_id")["abundancia"].var().sort_values(...).head(5)`, graficado con `px.box(..., color="physiological_state")`

D5.11 — Módulo 2 (Predictor): reproducción exacta del preprocesamiento de E2

Código: `data_access.py`, función `load_predictor_data` — `matriz.apply(_zscore)` (scipy), con `if slug in IMPUTA_CON_CERO: matriz = matriz.fillna(0)` aplicado antes del z-score (`IMPUTA_CON_CERO = {"laupheimer2024"}`), orden correcto y dataset correcto.

La columna `species`, para Ghosh2022 se completa vía el bucle `if c.startswith("species_")`: sobre las columnas faltantes del clasificador.

D5.12 — Módulo 2: predicción sobre muestra existente

Código: la ruta `/predictor` de `app.py` solo acepta `request.args.get("muestra")` sobre `matriz.index` (muestras ya existentes)

D5.13 — Módulo 3 (SHAP): simplificación

Código: `shap.TreeExplainer(modelo) + promedio de np.abs(shap_values)` sobre todas las clases (manejando los 3 formatos posibles de salida de SHAP: lista de arrays, array 3D, array 2D), etiquetado final por `name_original`

D5.14 — Módulo 4 (Generador de hipótesis): diseño y extensión a las 7 especies

Código: `CATEGORIAS_CONDICION` tiene una entrada por cada uno de los 7 slugs de `CVAE_SPECIES`, con el orden de categorías exacto que cada notebook de E3 usó en su `pd.Categorical(..., categories=[...])`. La función `construir_vector_condicion` replica `pd.get_dummies(drop_first=True)` a mano, un vector de ceros salvo un 1 en la posición de la categoría objetivo, exactamente el mismo mecanismo que E3 usó para condicionar el CVAE. `hipotesis.html` itera `{% for c in categorias %}` en el desplegable

D5.15 — `custom_objects` para la capa Lambda (sampling)

Código: `data_access.py` define `_sampling(args)` y la pasa como `custom_objects = {"sampling": _sampling}` en las 3 llamadas a `keras.models.load_model()` (encoder, decoder, autoencoder). La función se llama `_sampling` en el dashboard pero se registra bajo la clave `"sampling"`.

D5.18 — Módulo 5 (Espacio latente)

Código: `compute_latent_space` reconstruye el vector de condición real de cada muestra (no una condición fija) antes de llamar a `encoder.predict(...)`, y usa `z_mean` (no una muestra de `z`) para el scatter, para una vista exploratoria estable sin el ruido del muestreo aleatorio de la reparametrización



D5.19 — Módulo 6 (Actualizador de modelo)

Código : flujo de dos pasos (accion=probar / accion=confirmar) con token UUID (guardar_token_csv, ruta_token_csv); ACTUALIZABLES excluye ghosh2022, comparación de F1 macro con StratifiedKFold + cross_val_score contra el modelo actual antes de permitir el reemplazo (aplicar_actualizacion solo escribe a disco if resultado["mejora"])

D5.25 — data_access.py tiene definiciones duplicadas de 5 nombres

Hallazgo: el archivo define dos veces, en bloques separados, los mismos nombres de nivel de módulo

Como Python ejecuta el módulo de arriba hacia abajo y cada def/asignación reemplaza a la anterior con el mismo nombre, la segunda copia de cada una es la que efectivamente corre.

Código: Se borraron las 5 definiciones viejas y se dejó una sola copia de cada una, reordenando el archivo para que las declaraciones de datos (CLASSIFIER_SPECIES, CVAE_SPECIES, CATEGORIAS_CONDICION, etc.) queden todas juntas al principio.

D5.26 — Limpieza de nombres de compuesto

Decisión: la segunda definición de load_eda_data y compute_shap_summary aplican una función nueva, limpiar_nombre_compuesto(), que no existe en la primera versión de esas funciones

```
def limpiar_nombre_compuesto(nombre):  
    limpio = re.sub(r'\s*\[^\]*\)', '', nombre)  
    limpio = re.sub(r'\?.*$', '', limpio)  
    limpio = re.sub(r'\[s_]*(Std)?[s_]*d+$', '', limpio, flags=re.IGNORECASE)  
    return limpio.strip()
```

Quita anotaciones de laboratorio del nombre del compuesto antes de mostrarlo. No toca pubchem_cid

D5.27 — Creación de una función de predicción para muestras nuevas cargadas por CSV (predecir_muestra_nueva)}

Decisión: data_access.py define predecir_muestra_nueva(slug, archivo_csv, especie_ghosh=None), que calcula el z-score de una muestra nueva usando la media y el desvío de las muestras ya existente y predice con el clasificador ya entrenado.

D5.28 — Módulo 2 (Predictor): se conecta la carga de archivo nuevo

Decisión: Se conecta la función predecir_muestra_nueva() (ya implementada en data_access.py) a la ruta /predictor de app.py mediante un método POST adicional, y se agrega al template predictor.html un formulario de carga de CSV (sample_id, pubchem_cid, abundancia), con selector de subespecie para el caso de Ghosh2022. El módulo



pasa así de aceptar solo muestras ya existentes en la base a aceptar también un archivo nuevo subido por el usuario.

D5.29 — Despliegue del dashboard en la web (Render)

Decisión: Además de correr en el entorno local del proyecto, el dashboard se publica en Render (plan gratuito), quedando accesible en una URL pública.

Motivo: Se prioriza que los tutores puedan revisar el dashboard de forma independiente, sin depender de una demostración en vivo ni de instalar el entorno local (Python 3.12, TensorFlow, dependencias del proyecto).

Aunque el máster aborda la productivización de modelos, no cubre el despliegue en un servicio de hosting como Render, así que esta etapa se resolvió de manera autodidacta a partir de la documentación oficial de la plataforma